RicherText

PROJECT BRIEF · OCTOBER 2026

A calm, fast place to write.

RicherText is a minimal rich text editor for professionals: discover it, pay, and start writing in under two minutes. A small product built end to end, from landing page to live deployment.

Live demo (Stripe sandbox): richertext--richer-text-55ffe.europe-west4.hosted.app

34
decision records, each with the options weighed

690+
automated tests: unit, rules, Functions, end to end

3 days
from first commit to production, test-first

EU
data and servers in Belgium and the Netherlands

THE PRODUCT

The problem

People who write every day (consultants, PMs, lawyers, founders) choose between heavy suites that are slow and cluttered, and Markdown tools that are fast but make non-technical people learn syntax and paste badly into email. RicherText sits in the gap: it opens instantly, formats like a word processor, and gets out of the way.

One complete path, end to end

01
Land
Statically rendered landing page; pricing is a section, not a page.

02
Sign up
Email and password; the Free plan opens straight away.

03
Pay
Stripe Checkout; access unlocks when the webhook lands.

04
Write
Editor with autosave every second, online or offline.

05
Manage
Card, plan and cancel in Stripe's Customer Portal.

Principles that broke ties

Never lose a keystroke
Data safety outranks every other editor concern. It decided autosave, the offline cache and the content format.

Fast beats featureful
Every feature has to justify its cost in load time and clutter. It's why images, tables and AI are out.

Small but complete
A narrow product that works end to end beats a broad one that half works: landing, payments and editor all ship.

Boring infrastructure
Managed, proven services for auth, payments and hosting, so the effort goes into the writing experience.

Who it's for

Individuals who write daily and pay for their own tools. They value speed, focus, reliability and clean output; they don't need collaboration, databases or templates. That one sentence removed most of the feature list.

SCOPE DECISIONS

What's in, what's out, and why

In v1

- Headings, bold, italic, underline, strike, lists, links, quotes
- Text size, six text colours, writing font per document (Geist or IBM Plex Serif)
- Undo / redo and keyboard shortcuts for everything
- Autosave with a quiet "Saved" indicator; works offline
- Clean copy and paste to and from Docs, Word, Outlook and Gmail
- Free plan, monthly and yearly subscriptions, Customer Portal
- Light and dark, desktop and phone

Deliberately out

- Collaboration, sharing and comments: not what this user needs
- Images, tables and embeds: cost load time and clutter
- Templates, AI and plugins: a different product
- Full-text search: needs an external index; titles filter instead
- Native apps: the web app works at phone width
- Export: moved to next steps to ship the core first

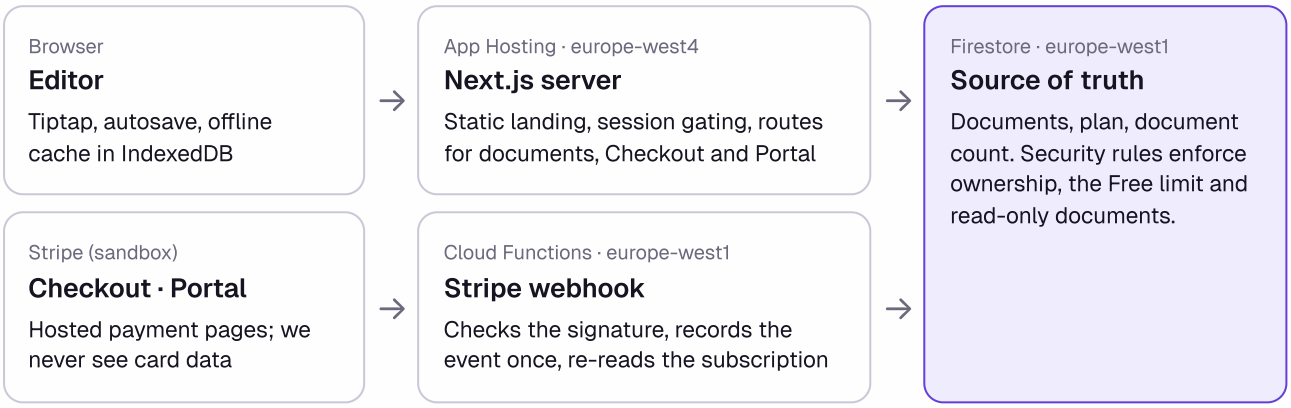
Business model choices

DECISION	WHY	ADR
Free plan instead of a trial	Full editor, one document at a time, no card. People feel the product before paying, and there's no trial expiry to engineer.	0020
Nothing is deleted on downgrade	When a paid plan ends, the newest document stays editable and the rest become read-only. Trust over pressure.	0020
Access during failed payments	Access continues while Stripe retries the card and locks only when it gives up. A declined card shouldn't lock someone out mid-sentence.	0027
Plan changes in the Portal	Card updates, monthly ↔ yearly and cancel all live in Stripe's Customer Portal: less code, fewer edge cases, the same UI customers know.	0027
No trials or promo codes	The Free plan does the trial's job; fewer states to keep consistent.	0027
No email verification in v1	Removes a step between landing and writing. Known cost: a mistyped email can't recover the account; it's first on the next-steps list.	0017
Sandbox payments only	A demo takes real Checkout flows without real charges; going live is a key swap plus tax handling.	0003

ARCHITECTURE

Few moving parts, one source of truth

The browser writes documents straight to Firestore, so its offline queue and retries protect every keystroke. Anything that must not be forged (creating documents, billing, the plan) goes through the server. Firestore is the only place that decides who has access.



Documents: browser → Firestore directly (rules decide). Create, delete and billing: browser → Next.js server → Firestore. Payments: Stripe → webhook → Firestore → the app updates live.

Key technical decisions

DECISION	WHY	ADR
Next.js on Firebase	One framework for a static, indexable landing page and a gated app; Firebase gives auth, database, functions and hosting under one project.	0001–2
Firestore	Security rules, an offline write queue and scale-to-zero match the principles; its weak spots (search, lock-in) don't matter here.	0004
Tiptap	Mature ProseMirror editor with MIT packages; one shared schema keeps paste, storage and rendering consistent.	0006
Session cookies	The client signs in; the server holds an httpOnly cookie and checks it in every layout, action and route, refreshed silently for active writers.	0026
Hand-written webhook	Signature-checked, idempotent (event IDs stored in the same transaction) and order-independent (re-reads Stripe), so retries and out-of-order events are safe.	0027
Server-side create and delete	Enforces the Free limit with a counter in a transaction; a query there serialised every user (8 parallel creates took 12 s; with the counter, 20 take 30 ms).	0028
Content as a JSON string	Firestore rejects maps nested deeper than 20 levels, so a list indented three times could never save. Found in an audit, fixed test-first.	0033
Offline cache in IndexedDB	Edits survive a reload or a dropped connection; shared by tabs and deleted on sign out.	0029

TRADE-OFFS

What I gave up, and why it was worth it

CHOSE	OVER	BECAUSE
Browser writes to Firestore	Saving through our own API	Firestore's offline queue and retries for free; rules do the checking. Cost: business rules live partly in security rules, so they get their own test suite.
Whole document per save	Sending only the edits	Simple and robust for memos of 10–40 KB. Edit-step syncing is collaboration machinery we don't need.
One Firebase project	Staging plus production	Less to run for a demo. Cost: rules, webhook and deploy changes can't be rehearsed in the cloud first, so the emulators and the deploy checks carry that weight.
Local deploy script	CI/CD and auto-deploys from GitHub	Every deploy runs the full suite from a clean main, with no extra service to secure. Cost: deploys depend on one machine.
Local first on the emulators	Building against a real project from day one	No cloud cost or secrets until the end, and fast, repeatable tests. Cost: production-only issues (regions, IAM, cleanup policies) surfaced at deploy time.
Stripe Portal for billing	Our own billing screens	Less code and fewer states. Cost: plan changes leave the app's design for a moment.
Scale to zero	An always-on server	Costs nothing while idle, within a €5 budget. Cost: a few seconds' cold start after a quiet spell; the landing page is served from the CDN anyway.
Geist only, one accent	A richer visual identity	A calm, typographic product; fewer decisions per screen. The writing font is the one place with a choice.

> Every trade-off was written down before building, with the options weighed, so the reasons survive longer than the conversation that made them.

HOW IT WAS BUILT

Test-first, decisions on paper, one design system

Tests first, then locked

Every change starts with a test that fails for the right reason. Existing tests can only change with explicit approval, so they hold the business rules in place.

Decision records

34 ADRs: context, options, decision, consequences. Superseded ones stay, so the history of a choice is readable.

Small vertical slices

Short-lived branches, conventional commits, merged only when lint, types and all tests pass in light, dark, desktop and phone.

What the tests cover

SUITE	COUNT	WHAT IT PROVES
Unit (Vitest)	416	Components, forms, autosave timing, paste clean-up, plan mapping, config guards, setup and deploy scripts
Emulator	103	Security rules (ownership, Free limit, read-only), webhook signatures and idempotency, deep nesting, no secrets in server logs
End to end (Playwright)	174	Every flow on desktop and phone: sign-up, checkout states, editor, offline, paste, landing without JavaScript, security headers, no secrets in the console

Security and privacy by default

- No secrets in the repo; real values only in Secret Manager
- Logs never carry tokens or personal data, checked by tests
- A production build refuses to run without real config
- Security headers on every response (no framing, HSTS)
- Billing data is server-written only; users read only their own
- Data and servers in the EU

DESIGN SYSTEM

Typography does the work

Large type and whitespace instead of graphics: no illustrations, no decorative elements. Landing page and editor share one atomic component set (tokens → atoms → molecules → organisms → templates), and nothing new appears on a screen until it's approved into the system.

Colour



Accent
#6440D8

Accent, dark
#B3A1FF

Text
violet-tinted

Muted text

Surface

Dark background

Type scale · 1.333

display	Aa
3xl	Write
2xl	Write calmly
xl	Write calmly
lg	Write calmly, fast
base	Body text, 16px on screen

Rules

- Geist everywhere; the logotype is Geist set italic and stretched
- One accent; colours only through theme tokens, in light and dark
- Spacing on a 4px grid: 4, 8, 12, 16, 24, 32, 48
- Soft radius: 5, 7 and 10px; flat surfaces
- Never place elements of different heights side by side
- Gaps inside a space are equal; edges one step larger
- Nothing changes size on state change

NEXT STEPS

What comes next

NEXT	WHAT AND WHY
Export	PDF and Word first, then RTF, Markdown, HTML and plain text; paid plans only, generated in a Cloud Function with the plan check. Already designed (ADR 0022); moved out of v1 to ship the core.
Email verification	Verify at sign-up and before checkout, so receipts and password resets reach the right person and throwaway sign-ups get harder.
Content-Security-Policy	A full policy for scripts, styles and connections to Firebase and Stripe, with nonces for Next's inline scripts. Basic headers are already live.
Staging environment	A second project with its own Stripe sandbox, so rules, webhook and deploy changes are rehearsed before users see them.
Automatic rollouts	Connect GitHub to App Hosting so merges to main deploy themselves, replacing the local script.
Performance and accessibility	Measure the editor against "interactive in under a second" on the live site; keyboard, screen-reader and contrast audit; a light and dark check of every screen.
Soft delete with Undo	Delete becomes reversible with an Undo toast instead of a confirmation dialog.
Account safety	Sign out of all devices; rate limits and abuse checks on document creation.
Paste checks in real apps	Verify paste and copy against the live Docs, Word, Outlook, Gmail, Pages and Notes, beyond the recorded samples.
Going live	Live Stripe keys, tax handling (Stripe Tax or a merchant of record), real reviews with permission, official brand logos and final legal copy.

Questions still open

Price points and the yearly discount (placeholders: $8 a month, $80 a year) · folders or tags · word count · full-text search · conversion and retention targets once there's traffic.